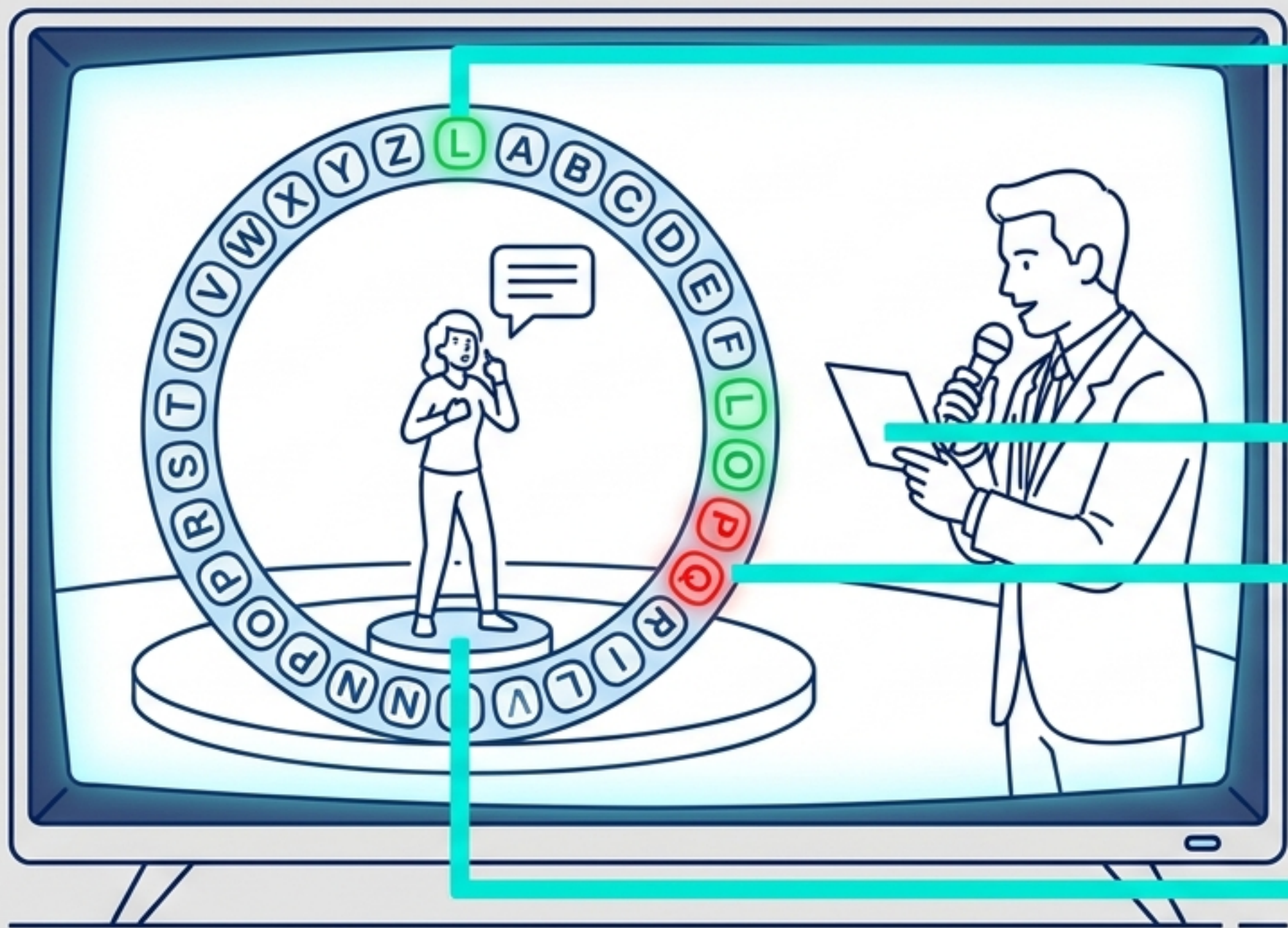


Los Planos del Rosco

Arquitectura C# y SQLite
para el juego definitivo
de palabras



Pregunta.letra

El dato puro

Pregunta.definicion

La pista

JuegoPasapalabra

El estado del motor

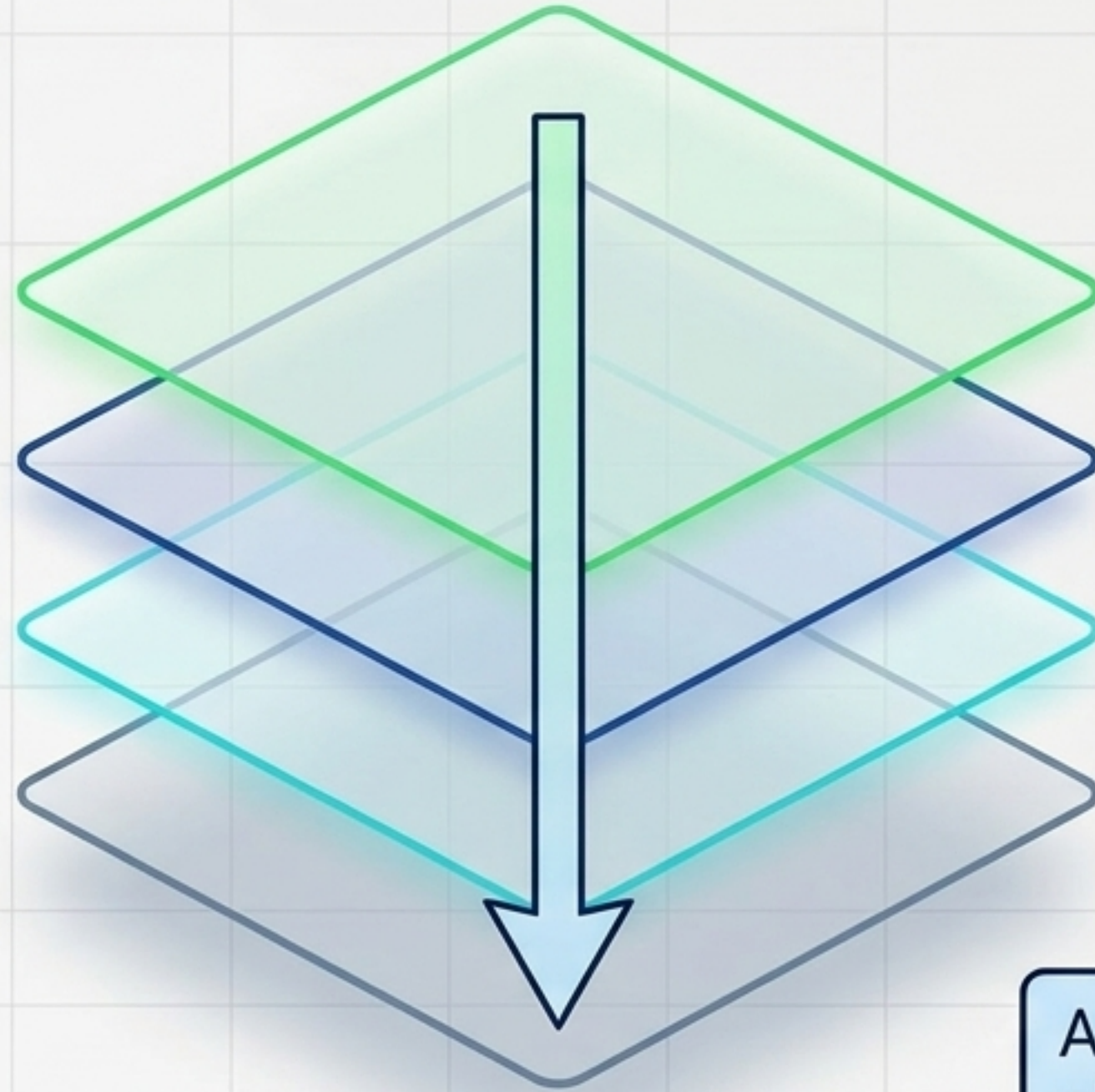
TextoUtil

El validador de cadenas

Cada elemento visual del programa esconde una responsabilidad única en nuestro software.

Separación en Capas: El Patrón Arquitectónico

- Capa 4: Lógica Core**
Clases: JuegoPasapalabra, TextoUtil
- Capa 3: Acceso a Datos**
Clases: GestorTemas, GestorPreguntas
- Capa 2: Entidades**
Clases: Tema, Pregunta
- Capa 1: Infraestructura**
Fichero: BaseDatos (SQLite)



- Capa 4: Lógica Core**
Clases: JuegoPasapalabra, JuegoPasapalabra, TextoUtil
- Capa 3: Acceso a Datos**
Clases: GestorTemas, GestorPreguntas
- Capa 2: Entidades**
Clases: Tema, Pregunta

Al separar el sistema, evitamos el código espagueti. Cada clase tiene un único trabajo.

La Bóveda de Datos: SQLite en Local



Ruta Relativa:
Se ejecuta en la misma carpeta carpeta que el programa, sin servidores externos.

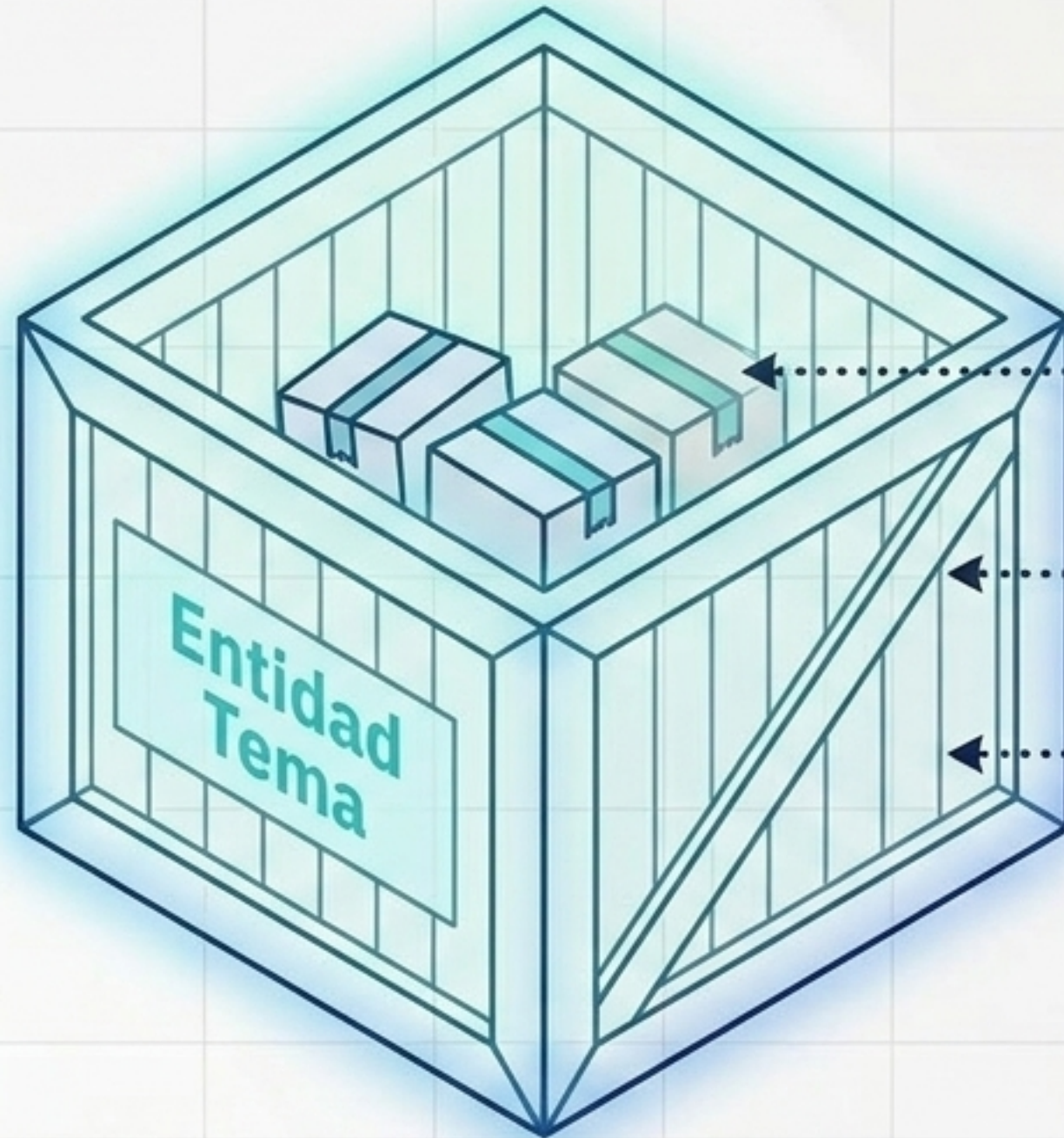
```
using Microsoft.Data.Sqlite;

class BaseDatos {
    // La base de datos se encuentra
    // en un único fichero.
    // Se buscará en la misma carpeta
    // desde la que se ejecute.

    private const string
        CadenaConexion = "Data
        Source=pasapalabra.db";
}
```

Fichero Único: Toda la información reside en un solo lugar.

Modelos (I): Estructurando los Temas



```
class Tema {  
    ➔ private int id;  
    ➔ private string nombre;  
    ➔ private string descripcion;  
}
```

- Un **modelo de datos** (Entity) no hace acciones; simplemente almacena la información que traemos de la base de datos.

Modelos (II): El ADN de una Pregunta



```
class Pregunta {  
    private int id;  
    private char letra;  
    private string respuesta;  
    private string definicion;  
    private tema;  
}
```

Una **pregunta** no solo tiene texto, tiene un contexto asociado. El modelo incluye otro modelo dentro de sí mismo.

Anatomía de Clases: Modelos vs. Gestores



Los Modelos
(Tema, Pregunta)



Los Gestores
(GestorTemas, ...)

Rol: El Contenedor de Datos

Rol: El Trabajador
(Data Access Object)

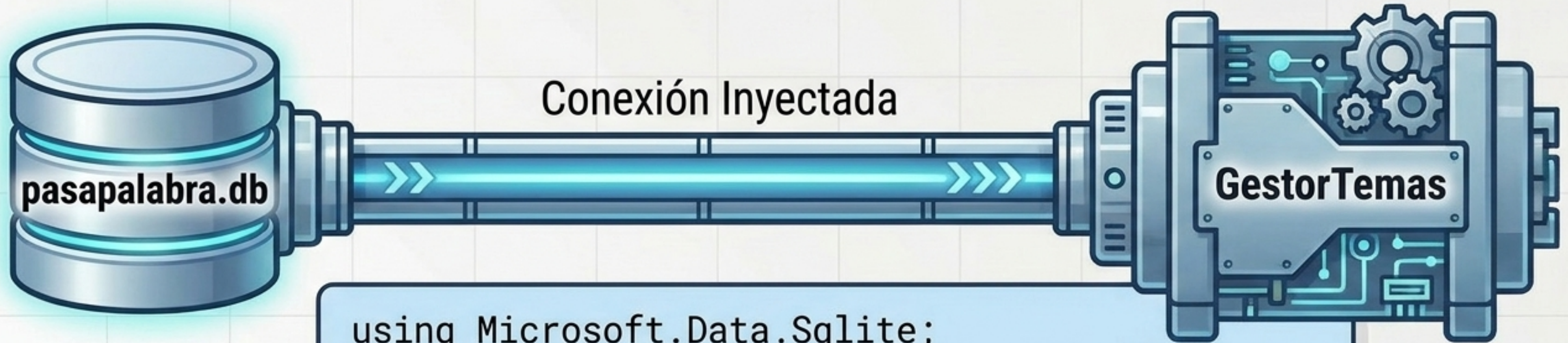
Atributos: Variables primitivas
(string, int)

Atributos: Conexiones a BD
(SqlConnection)

Metáfora: La carta física con la
pregunta escrita

Metáfora: El azafato que busca
la carta en el archivo

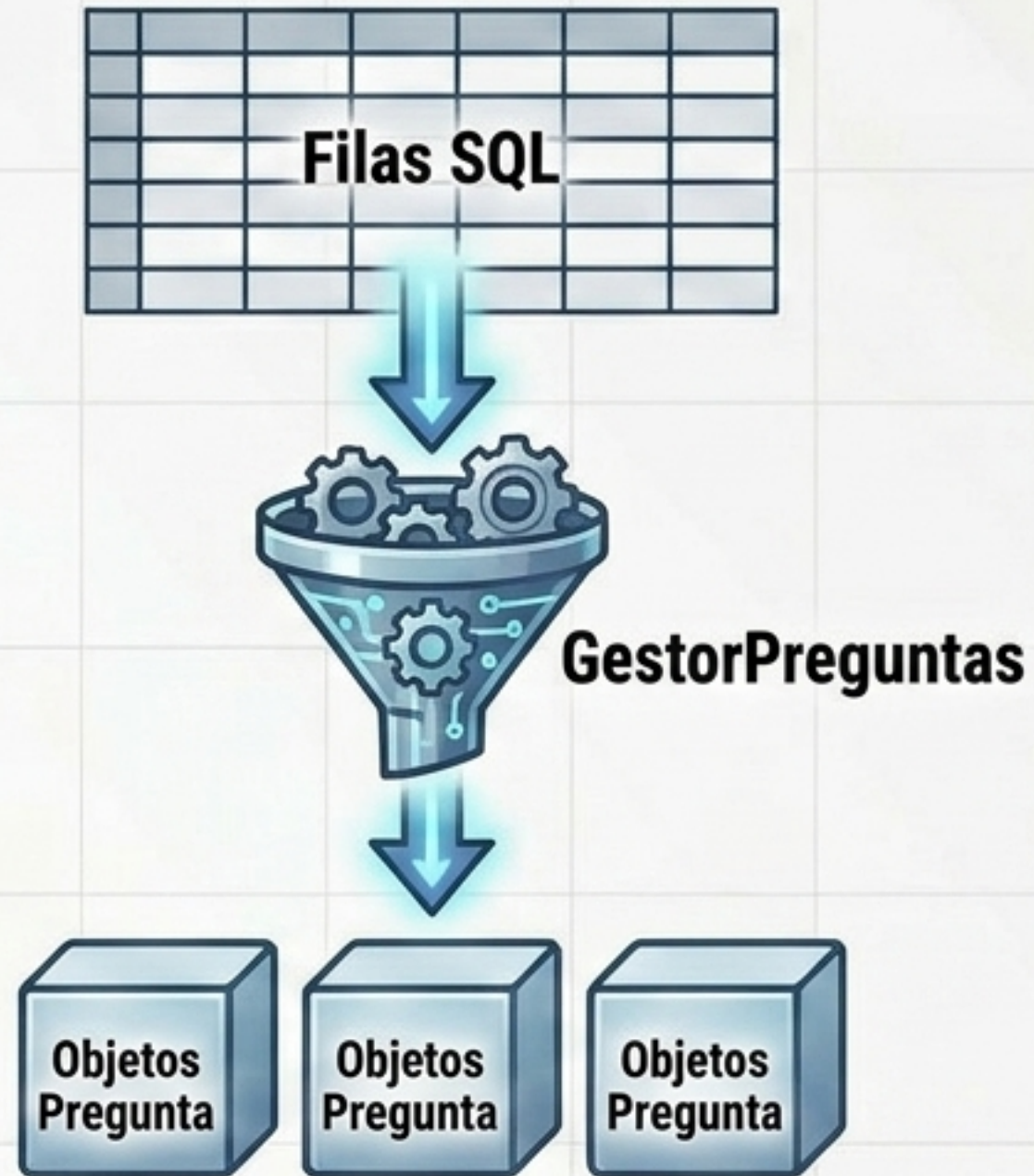
El Gestor de Temas: Acceso a Datos



```
using Microsoft.Data.Sqlite;  
  
class GestorTemas {  
    private readonly SqliteConnection conexion;  
}
```

El **gestor** necesita la **conexión** para hacer su trabajo. Actúa como el puente exclusivo entre la lógica de tu programa y las tablas de SQLite.

El Gestor de Preguntas: Sirviendo el Rosco



```
using Microsoft.Data.Sqlite;  
  
class GestorPreguntas {  
    private readonly SqliteConnection conexion;  
}
```

Separar los gestores permite que el código sea **mantenible**. Si mañana cambiamos SQLite por otra base de datos, solo tocamos estos ficheros.

Lógica Core: El Motor del Pasapalabra



```
using Microsoft.Data.Sqlite;  
  
class JuegoPasapalabra {  
    private readonly SqliteConnection conexion;  
}
```

Aquí reside **el estado de la partida**: qué letra toca, cuántos aciertos hay, y la validación del turno. Es el director de la orquesta.

El Desafío del Español: Validando Respuestas

Input del Jugador:
" canción "



Respuesta en BD:
"cancion"

False (Fallo Injusto)

Un **espacio extra** o una **tilde** no deberían arruinar el rosco.
Necesitamos sanitizar los datos de entrada.

Utilidades: Normalización de Cadenas

Cadena Original

" El Avión "

Trim & Split

El Avión

Quitar Tildes

El Avion

To Lower

el avion

```
using System.Text;
```

```
class TextoUtil  
{
```

```
    public static string NormalizarParaComparar(string texto)  
    {
```

```
        string textoSinEspaciosExtra = string.Join(  
            " ", texto.Trim().Split(' ', StringSplitOptions.RemoveEmptyEntries));
```

```
    } // ...
```

```
}
```

Método `static`:

No requiere instanciar objeto

El Punto de Entrada: Arrancando el Sistema

```
using Microsoft.Data.Sqlite;
using System.Text;

class Program {
    static void Main() {
        Console.OutputEncoding = Encoding.UTF8;
        Console.InputEncoding = Encoding.UTF8;
        // Iniciar juego...
    }
}
```

Vital para el Español:
Sin esta configuración,
la consola de Windows
no mostrará ni leerá
correctamente las 'ñ'
o las tildes.

Program.cs actúa como el interruptor general.

Ciclo de Vida: El Viaje de un Turno



Arquitectura Completada: El Rosco Funciona

El buen software, como un buen juego, se construye pieza a pieza.

